



A Physic-Informed Neural Network Approach to Orbit Determination

Andrea Scorsoglio¹ · Luca Ghilardi¹ · Roberto Furfaro¹ 

Accepted: 10 July 2023 / Published online: 8 August 2023

© The Author(s), under exclusive licence to American Astronautical Society 2023

Abstract

This paper introduces a method for solving orbit determination problems named Physics Informed Orbit Determination. We use a particular kind of single-layer, feed-forward neural network with random input weights and biases called Extreme Learning Machines to estimate the spacecraft's state. The least-squares estimate is used as the baseline for the loss function, to which a regularizing term based on the differential equations modeling the dynamics of the problem is added. This ensures that the learned relationship between input and output is compliant with the physics of the problem while also fitting the observation data. The method works with range/range-rate or angular observations, either in Keplerian or non-Keplerian dynamics. The method is tested on synthetically generated data, with and without perturbations. The results are comparable with the batch least-squares solution, with the advantage of not requiring an initial guess and solving for the entire trajectory without any integration.

Keywords Orbit determination · Machine learning · Physics informed neural networks

1 Introduction

The problem of orbit determination (OD) dates back to Newton and Gauss. Their early studies helped describe the orbits of various comets as well as the dwarf planet Ceres. Currently, the success of virtually any mission is related to the possibility of

✉ Roberto Furfaro
robertof@arizona.edu

Andrea Scorsoglio
andreascorsoglio@arizona.edu

Luca Ghilardi
lucaghilardi@arizona.edu

¹ Department of Systems & Industrial Engineering, University of Arizona, 1127 E. James E. Rogers Way, Tucson, AZ 85721, USA

estimating the state and pose of the spacecraft at any given time. For instance, it is crucial to estimate the state of a spacecraft as soon as it leaves the launcher to assess the steps to take during the commissioning phase. Furthermore, reliable and accurate OD algorithms are pivotal for planning correction maneuvers and station-keeping tasks and for missions involving relative motion, like rendezvous and docking. Also, OD is important in the field of space situational awareness, to track potentially hazardous man-made and natural objects in the vicinity of Earth and Moon.

Historically, OD and state estimation have been solved by processing a series of observations of an object using analytical or statistical methods. The analytical methods are generally referred to as Initial Orbit Determination (IOD) methods, while statistical methods are commonly divided into batch least squares methods and Kalman filters (KF). For objects in geocentric orbits, especially if in Low Earth Orbit (LEO), the measurements needed for state estimation can come from various sources and are generally fairly simple to obtain, such as onboard Global Positioning System (GPS) or ground-based observations. In fact, at low altitude it is possible for cooperative objects to obtain accurate position measurements from GPS satellites that can then be used on-board directly and/or forwarded to the ground. The conditions in LEO are also favourable for optical or radar measurements from ground stations, which can provide angle (right ascension/declination) or range/range-rate respectively, both for cooperative and non-cooperative objects. Both IOD and statistical methods can be used in this case. Common IOD methods are Gauss' Method [1], Double-R Iteration [2], or Gooding's Method [3]. These methods require a set of three angular measurements to estimate an orbit.

On the other hand, statistical OD methods use an extensive collection of measurements to refine the estimate, starting from initial state and covariance estimates. Batch algorithms, commonly referred to as batch least-squares [4–6], can estimate the initial state of the object by using the entirety of the available observations simultaneously. They work by estimating a set of constant weights multiplied by a predefined set of basis functions, providing a linear estimate of the measure. This has been extended to allow for non-linear relationships between parameters and measures: the resulting algorithm is usually referred to as non-linear least squares or sequential least squares.

Sequential algorithms instead can estimate the current state of the object based on previous knowledge, with the ability of continually refining the estimate as new observations are gathered [4, 5]. They are normally referred to as KF [7] and their variants [8]. They generally work by estimating the state distribution's probability density function (pdf). In the case of linear systems, this is a Gaussian distribution, which is fully determined by its mean vector and covariance matrix. The mean and covariance are updated analytically within the KF algorithm. The algorithm is often used for non-linear systems by linearizing the system's process and measurement models. Different ways of linearizing the models lead to different filters, namely the extended Kalman filter (EKF), the iterated extended Kalman filter (IEKF) [9] and the Linear Regression Kalman Filter (LRKF) [10].

All the methods described above are commonly used together and often sequentially to refine the orbit estimate. A typical application scenario requires a track initialization using IOD methods, which is then used as the initial guess for the batch

least squares, whose solution is then used as an initial reference trajectory for a KF OD method. This procedure is not straightforward and requires good knowledge of the measurements' covariance to work properly.

However, in the cislunar space, the standard techniques of OD applied to near-Earth objects cannot be applied. This is due to the relative scarcity of observations for non-cooperative objects in such areas, and the non-negligible third-body effects. For missions towards the Moon, what spacecraft have used in the past is a combination of ground-based initial state updates and inertial measurements. Although these techniques are powerful, they inherently suffer from error drift due to numerical integration [11]. With cooperative objects in the cislunar space, i.e., the ARTEMIS spacecraft, orbit determination is obtained currently using a batch least-squares method that processes range and Doppler tracking measurements from the NASA Deep Space Network (DSN). This is also the method used to track deep-space satellites, even at distances past the Moon's orbit. Although powerful, this method cannot be applied to non-cooperative objects. For non-cooperative objects, orbit determination can only be done using optical observations, given the unavailability of ranging measurements. However, techniques based on optical observations that work well for Keplerian orbits do not usually work for objects whose dynamics are described by the circular restricted three-body problem (CRTBP). **Most orbit determination techniques can only estimate the object's state for short periods, close to the observation epoch.** Moreover, in this dynamical regime, frequent maneuvering is needed to maintain the orbit, which cannot be accounted for in the OD process if they are not known, and for this reason, degrades its capabilities even further.

This paper builds a method based on neural networks (NN) [12] to solve the OD problem. Examples of machine learning and neural networks being applied to OD problems are present, but the field is relatively new. Furfaro et al. [13] demonstrated that it is possible to create an accurate inverse mapping between angle measurements and orbital parameters using Extreme Learning Machines (ELM). Sharma [14] presented an algorithm for orbit determination and magnetometer interference cancellation for CubeSats, demonstrating robustness in a variety of scenarios (e.g., Doppler-only, position-based, lunar orbit, and LEO). Lee et al. [15] presented a new interesting way to simulate range rate and slant range observations using machine learning. Peng et al. [16] also demonstrated the viability of machine learning, specifically Artificial Neural Networks (ANNs), for orbit prediction accuracy improvement.

The idea is to build upon the ML methods applied to the OD problem using the recently developed Physics-Informed Neural Networks (PINN) [17] to regularize the learning algorithm and improve the dynamical soundness of the estimated solution. They were developed for solving forward and inverse problems governed by parametric differential equations (DEs). The training of these networks is regulated by the physics of the problem. In particular, the latent solution of the parametric DE is approximated via NNs; the DE that models the physics of the problem is then embedded, in its implicit form, into the NN loss function. Thus, the DE drives the network's training, acting as a regulator that penalizes the network's training when the physics of the problem is violated. In this case the method employs a single layer feed-forward neural network (SLFN)

with random input weights and biases to approximate the state of the satellite. This kind of NN was first introduced in 1992 by Schmidt et al. [18] as a neural network with a single hidden layer of neurons, whose input weights and biases are randomly initialized at the start of training and never changed. This peculiar structure allows training to be carried out using any least-squares technique instead of back-propagation since the only trained parameters are the output weights. Huang et al. then formalized this kind of NNs under the name ELM [19]. In the field of PINNs, ELM were then used to derive the method named Physics Informed Extreme Learning Machines (PIELM) [20], which has been used to solve a variety of problems from high-speed flow [21] to cardiac activation mapping [22]. These methods have also been extended using a functional interpolation technique called Theory of Functional Connections (TFC) [23] to analytically satisfy the constraints of the problem, leading to an improvement in the accuracy of the solution, leading to a variety of PINN–TFC frameworks [24, 25].

In this work, LEO, geostationary orbits (GEO), and Molniya Orbits are considered for Keplerian dynamics, while Lyapunov and halo orbits for non-Keplerian dynamics. The regularizer part of the loss function is defined by the two-body or circular restricted three-body problem (CRTBP) equations of motion that govern the orbits mentioned above. The part of the loss related to the data is instead defined using either range/range-rate or angular observations in the form of right ascension and declination, defined in the Earth centered inertial frame. This allows the network to solve the OD problem in both dynamical frameworks.

The results show performance similar to the batch least squares in terms of estimation accuracy, with a key advantage: it does not require an accurate initial guess. The only guess needed are input weights and biases of the ELM, which by definition can be initialized at random. Batch least squares instead requires the initial guess to be reasonably close to the true state to be able to converge. This paper also shows how flexible a PINN-based OD system can be as it can adapt to multiple dynamics and measurement models. In particular, the ability to work with angle-only measurements coming from optical telescopes is interesting if the mission requires estimating the state of uncooperative objects.

The remainder of the paper is organized as follows: Sect. 2 introduces the concepts of classical batch least squares and physics informed neural networks on which the method is based; Sect. 3 describes the problem to be solved and the dynamical and measurement models; Sect. 4 describes the method introduced in this paper; Sect. 5 summarizes the numerical results obtained in a variety of OD scenarios and finally Sect. 6 concludes the paper with some final remarks.

2 Background

In this section, building blocks of the physics-informed orbit determination techniques are introduced: the classical orbit determination problem and the physics-informed neural networks in the form of PIELM.

2.1 The Orbit Determination Problem

The OD problem is typically cast as an estimation problem where a set of measurements of the same or a collection of quantities related to the problem are used to estimate the system's state. The significant techniques in this field are Least Squares (and Batch Least Squares) estimate and Kalman Filters. We focus on the first group, in this case, as a foundational concept to create our OD framework and the comparison to our method. According to least squares, suppose there is a set, or batch, of measured values $\tilde{y}_j$, taken at discrete time instants t_j :

$$\{\tilde{y}_1, t_1; \tilde{y}_2, t_2; \dots; \tilde{y}_m, t_m\} \quad (1)$$

and a mathematical model correlating the measurement with some constants x_i :

$$y(t) = \sum_{i=1}^n x_i h_i(t), \quad (2)$$

where $h_i(t)$ are a set of basis functions. The x_i are a set of constants whose numerical values are unknown. In classical least squares, we seek a set of estimates, denoted by $\{\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n\}$, which can be used to predict $y(t)$. In this case, we used the weighted least squares, which is obtained by considering the following objective function [4]:

$$J = \frac{1}{2} \mathbf{e}^T \mathbf{W} \mathbf{e}, \quad (3)$$

where $\mathbf{W}$ is a diagonal weighting matrix and $\mathbf{e}_j = \tilde{y}_j - \hat{y}_j$ is the residual error between the measured and predicted measurement. The solution is:

$$\hat{\mathbf{x}} = (\mathbf{K}^T \mathbf{W} \mathbf{K})^{-1} \mathbf{K}^T \mathbf{W} \tilde{\mathbf{y}}, \quad (4)$$

where $\mathbf{K} = \frac{\partial f_m(\mathbf{x})}{\partial \mathbf{x}}$ is the Jacobian on the measurement model.

Regardless of the models used, the dynamical and measurement models can be defined generally as $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}(t))$ and $f_m(\mathbf{x})$ respectively. The non-linear nature of the problem requires the usage of a non-linear least squares estimation [5]. This is optimized by linearizing the model around a reference trajectory and solving for the variations instead of the real state. This works by initializing the vector $\mathbf{x}_0$ with an initial guess. Given this initial guess, generate the reference trajectory by integrating the equations of motion and simultaneously integrating the State Transition Matrix (STM) using $\dot{\Phi} = \mathbf{A}(t)\Phi$, with $\mathbf{A}(t)$ being the linearized system matrix and initial conditions $\Phi(0, 0) = \mathbf{I}$. The estimated state can be expressed as:

$$\hat{\mathbf{x}} = \mathbf{x}_c + \delta \mathbf{x} \quad (5)$$

The measurement model is:

$$f_m(\hat{\mathbf{x}}) \approx f_m(\mathbf{x}_c) + \mathbf{K} \delta \mathbf{x}, \quad (6)$$

where $\mathbf{K} = \frac{\partial f_m}{\partial \mathbf{x}}|_{\mathbf{x}_c}$ is the Jacobian calculated over the reference trajectory. The observations variation is:

$$\delta \mathbf{y} = \tilde{\mathbf{y}} - f(\hat{\mathbf{x}}) \approx \tilde{\mathbf{y}} - f(\mathbf{x}_c) - \mathbf{K} \delta \mathbf{x} = \delta \mathbf{y}_c - \mathbf{K} \delta \mathbf{x}, \quad (7)$$

where $f(\mathbf{x}_c)$ are the observations computed on the reference trajectory. The objective function is then:

$$J = \frac{1}{2} \delta \mathbf{y}^T \mathbf{W} \delta \mathbf{y} = (\delta \mathbf{y}_c - \mathbf{K} \delta \mathbf{x})^T \mathbf{W} (\delta \mathbf{y}_c - \mathbf{K} \delta \mathbf{x}) \quad (8)$$

The solution for the initial state variation is:

$$\delta \hat{\mathbf{x}} = (\mathbf{K}^T \mathbf{W} \mathbf{K})^{-1} \mathbf{K}^T \delta \mathbf{y} \quad (9)$$

The initial state estimate is then updated:

$$\hat{\mathbf{x}}_0^{k+1} = \hat{\mathbf{x}}_0^k + \delta \hat{\mathbf{x}} \quad (10)$$

and a new reference trajectory computed. This is iterated until the user defined convergence criterion $\|\hat{\mathbf{x}}_0^{k+1} - \hat{\mathbf{x}}_0^k\| \leq \epsilon$ is met.

2.2 Physics Informed Extreme Learning Machines

The method introduced in this paper is a modified least-squares that leverages the capabilities of PIELM [20]. In order to properly define PIELM, it is important to introduce PINN, as PIELM are a particular subset of them and share the same basic idea. PINNs were first introduced by Raissi et al. [17] as a method for approximating solutions to general non-linear PDEs, as well as inverse problems. Unlike mesh-based methods (such as the finite elements method), PINN uses a deep neural network (DNN) as a basis function to approximate the solution. First, the PDE and its boundary conditions are sampled on locations within and at the domain's boundaries. Then, the output of the DNN at those locations is used to compute the residuals of the PDE on the internal points and the boundaries. The loss is then expressed as the summation of the errors on the known boundary conditions and the residuals based on the physical relationships of the problem. The weights and biases of the neural network are then learned by minimizing the loss using a gradient-based optimization method. It should be noted that, although PINNs were introduced for solving PDEs, they can also be employed to solve ODEs and systems of ODEs where the only input to the network is time and the boundary conditions are not defined [24]. This has been shown to work well in a variety of continuous and discrete-time models [17]. In mathematical terms, PINN can be described by considering a general problem in the form:

$$u_t + N[u] = 0, \quad x \in \Omega, \quad t \in [0, T], \quad (11)$$

where u is the latent solution and $N[\cdot]$ is any nonlinear differential operator. We then define $f(t, x)$ as the left-hand side of Eq. (11):

$$f(t, x) = u_t + N[u] \quad (12)$$

The main idea of PINN is to approximate the solution $u(t, x)$ with a neural network. The neural network then becomes a physics informed neural network by training its weights and biases so that they minimize the mean squared error loss:

$$L = MSE_u + MSE_f, \quad (13)$$

where

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2 \quad (14)$$

and

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2, \quad (15)$$

where $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ denote the initial and boundary training data on $u(t, x)$ and $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ denote the collocation points within the domain for $f(t, x)$. This loss ensures that the solution respects both the dynamics and the initial/boundary conditions.

PIELM is a hybrid neural network-based method that integrates the concepts of PINNs, explained above, and the efficiency of ELM [19]. ELMs are a particular kind of single-layer feed-forward networks in which the input weights and biases are randomly chosen and constant through the learning process while the output weights are the only trained parameters. This allows the training to be carried out in a single step, drastically decreasing computational time. ELM also have shown to have outstanding representation power and better generalization than classical gradient-based learning procedures [26]. In mathematical terms, considering a single layer feed forward network with n neurons, $\mathbf{x}$ is the input state and y is the output, an ELM is a mapping between input and output through a series of non-linear basis functions called activation functions σ and a set of parameters called input weights ($\mathbf{w}$), biases ($\mathbf{b}$) and output weights (β). The output of a single neuron is $\sigma(\mathbf{w}_j^T \mathbf{x} + b_j)$. For a given dataset $\{\mathbf{x}_k, y_k\}_{k=1}^N$ with N samples, the output is given by:

$$\mathbf{y} = \mathbf{H}(\mathbf{x})\beta, \quad (16)$$

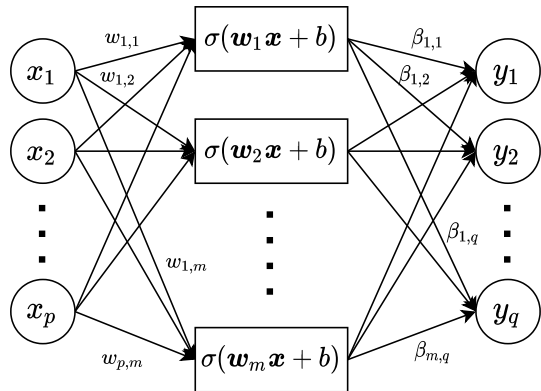
where

$$\mathbf{H} = [\mathbf{h}(\mathbf{x}_1), \mathbf{h}(\mathbf{x}_2), \dots, \mathbf{h}(\mathbf{x}_T)]^T \quad (17)$$

is the output of the hidden layer, with

$$\mathbf{h}(\mathbf{x}_k) = [\sigma(\mathbf{w}_1^T \mathbf{x}_k + b_1), \sigma(\mathbf{w}_2^T \mathbf{x}_k + b_2), \dots, \sigma(\mathbf{w}_n^T \mathbf{x}_k + b_n)] \quad (18)$$

Fig. 1 ELM architecture



Training is achieved by minimizing the error between the predicted output $\mathbf{y}$ and the real output $\mathbf{T}$, $\epsilon = \mathbf{y} - \mathbf{T}$:

$$J = \frac{1}{2N} \epsilon^T \epsilon \quad (19)$$

In the classical ELM definition, the solution is obtained analytically as:

$$\beta = \tilde{H}T, \quad (20)$$

where $\tilde{H}$ is the pseudoinverse of matrix H , which is in general not squared. Figure 1 shows a representation of an ELM.

The PIELM learning procedure reduces to choosing an ELM as a parametric representation of the state of the PDE/ODE and learning its parameters by optimizing a physics-based loss function, i.e., the summation of PDE and BCs least-squares cost as explained above.

3 Problem Description

In this paper, the problem of estimating the state of a satellite given a sequence of observations is solved using PIELM. The object is considered either in a Keplerian or a non-Keplerian orbit. Specifically, we consider LEO, GEO, Molniya orbits as Keplerian orbits, and Lyapunov and Halo orbits around the L2 lagrangian point in the Earth–Moon system as non-Keplerian orbits. The satellite dynamics models are the Two-Body Problem for Keplerian orbits and the CRTBP for non-Keplerian orbits. Moreover, the problem is solved for all orbits using either range/range-rate and right ascension and declination (α and δ) as measurements. This section describes the dynamical and measurement frameworks in more detail.

3.1 Unperturbed Two-Body Problem

In the two-body problem, two bodies are considered orbiting around the system's center of mass. The equations of motion describe the relative motion of the second, smaller body around the center of mass of the system, subject to the gravitational attraction between the two. Perturbations, such as solar radiation pressure (SRP), atmospheric drag, third body effect, and non-uniform mass distribution of the Earth can be added as additional forcing terms in the equations of motion, but are not considered in our preliminary study. The equations are:

$$\begin{cases} \ddot{x} = -\frac{\mu}{r^3}x \\ \ddot{y} = -\frac{\mu}{r^3}y \\ \ddot{z} = -\frac{\mu}{r^3}z, \end{cases} \quad (21)$$

where $r = \sqrt{x^2 + y^2 + z^2}$ is the radial distance of the object from the origin.

3.2 Circular Restricted Three-Body Problem

The simplest dynamical model that considers two main attractors is the restricted circular three-body problem framework (CRTBP). The two larger bodies are considered primaries. We are interested in the motion of a third small body in the vicinity of the two primaries. The problem dynamics are generally expressed in the absolute synodic reference frame. The origin of this frame is positioned in the center of mass of the system G; the x -axis is aligned with the line connecting the two primaries, the z -axis is parallel to the angular momentum vector of the primaries, and the y -axis completes the orthonormal triad. The frame rotates with an angular velocity equal to the mean angular motion of the two primaries around their center of mass. Moreover, quantities in this reference frame are made non-dimensional by introducing a unit of length and a unit of time. The only parameter governing the dynamics of the system is the mass parameter $\mu = \frac{m_2}{m_1 + m_2}$, with m_1 and m_2 being the masses of the two primaries. In this reference system, the equations of motion describing the dynamics of the particle are the following:

$$\begin{cases} \ddot{x} - 2\dot{y} = x - \frac{1-\mu}{r_1^3}(x+\mu) - \frac{\mu}{r_2^3}(x-(1-\mu)) \\ \ddot{y} + 2\dot{x} = y - y\left(\frac{1-\mu}{r_1^3} + \frac{\mu}{r_2^3}\right) \\ \ddot{z} = -z\left(\frac{1-\mu}{r_1^3} + \frac{\mu}{r_2^3}\right) \end{cases} \quad (22)$$

with

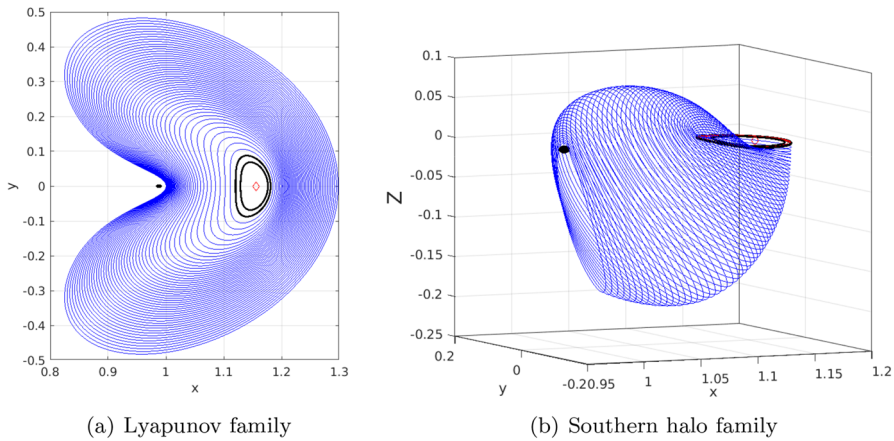


Fig. 2 CRTBP orbits

$$\begin{aligned} r_1 &= \sqrt{(x + \mu)^2 + y^2 + z^2} \\ r_2 &= \sqrt{(x - (1 - \mu))^2 + y^2 + z^2} \end{aligned} \quad (23)$$

More details can be found in Koon et al. [27]. Equations (22) do not have a closed-form, analytical solution. However, it is possible to determine the location of equilibrium points of the CRTBP. The equilibrium points, also named Lagrange points, are stationary points of the potential function U defined as

$$U = \frac{1}{2}(x^2 + y^2) + \frac{1 - \mu}{r_1} + \frac{\mu}{r_2} \quad (24)$$

and are the solutions of the equation

$$\nabla U = 0 \quad (25)$$

In this framework, various trajectories exist that result in a periodical motion around the Lagrange points. They can be divided into two main groups: in-plane, Lyapunov, and out-of-plane, or halo, orbits. The two families taken into consideration in this paper are Lyapunov and southern halo orbits around the L2 point, shown in Fig. 2.

3.3 Range/Range-Rate Observations

Range and range rate observations are a very common type of measurement for OD. The observations are generally obtained using Δ -DOR (Delta-Differential One-Way Ranging) using a radio-telescope. Generally they must come from different observatories to be able to determine the state of the system. The measurement model, as function of the spacecraft state $\mathbf{x} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]$ and the position of the i -th observatory $\mathbf{r}_i = [x_{s_i}, y_{s_i}, z_{s_i}]$ is:

$$\begin{aligned}\rho_i &= [(x - x_{s_i} \cos \theta + y_{s_i} \sin \theta)^2 + (y - x_{s_i} \sin \theta - y_{s_i} \cos \theta)^2 + (z - z_{s_i})^2]^{1/2} \\ \dot{\rho}_i &= [(x - x_{s_i} \cos \theta + y_{s_i} \sin \theta)(\dot{x} + x_{s_i} \sin \theta \dot{\theta} + y_{s_i} \cos \theta \dot{\theta}) \\ &\quad + (y - x_{s_i} \sin \theta - y_{s_i} \cos \theta)(\dot{y} - x_{s_i} \cos \theta \dot{\theta} + y_{s_i} \sin \theta \dot{\theta}) + (z - z_{s_i})\dot{z}] / \rho_i,\end{aligned}\quad (26)$$

where θ and $\dot{\theta}$ are the angular rotation and rotation rate of the Earth-Centered, Earth-Fixed (ECEF) frame to the Earth-Centered Inertial (ECI) frame, respectively.

3.4 Angular Measurements

Angular measurements usually refer to any measure of the position of an object in the sky, regardless of its range. This is obtained from astrometric calculations using optical observations. The position is generally defined in the local sky frame, in which case azimuth and elevation are considered, or the inertial sky, in which case right ascension (α) and declination (δ) are considered. In this case, the observations are considered to be topocentric right ascension and declination defined in the local inertial frame. Moreover, to avoid indetermination in the definition of the inverse trigonometric function and to avoid jumps in the measures, we consider sine and cosines of the angles instead of the angles themselves. The measurement model is then:

$$\begin{aligned}\sin \alpha &= \frac{y - y_s}{r} \\ \cos \alpha &= \frac{x - x_s}{r} \\ \sin \delta &= \frac{z - z_s}{r}\end{aligned}\quad (27)$$

where $r = \sqrt{(x - x_s)^2 + (y - y_s)^2 + (z - z_s)^2}$ and $r_{xy} = \sqrt{(x - x_s)^2 + (y - y_s)^2}$.

4 Physics Informed Orbit Determination (PIOD)

This paper aims to approximate the state of the satellite using an ELM as a function of the epoch t . Specifically, we want to estimate the whole state of the system $\mathbf{x} = [\mathbf{r}, \mathbf{v}]$, comprising of position and velocity. Due to the structure of a single layer network, we can achieve that by creating a network whose output is the spacecraft's 3D position $\mathbf{r}$ and obtain the velocity by analytically differentiating the output. So the position of the satellite is expressed as:

$$\mathbf{r} = \text{ELM}(t, \boldsymbol{\beta}) \quad (28)$$

where t is the epoch since the first observation and $\boldsymbol{\beta}$ are the trainable parameters of the network. We initialize the input weights $\mathbf{w}$ and biases $\mathbf{b}$ as random vectors between -1 and 1 of length m , where m is the number of neurons. The output weights $\boldsymbol{\beta}$ are also initialized as a random vector of length $3m$ between 0 and 1.

The matrix $\mathbf{H}$, the output of the hidden layer of the network as seen in Eq. (17), is obtained, along with its derivatives ($\dot{\mathbf{H}} = \frac{d\mathbf{H}}{dt}$, $\ddot{\mathbf{H}} = \frac{d^2\mathbf{H}}{dt^2}$), using a hyperbolic tangent activation function. It is important to note that, since the activation function is known, its derivative are computed analytically. The output of the net (i.e. the position of the satellite) and its derivatives are then:

$$\mathbf{r} = [r_1, r_2, r_3] = [\mathbf{H}^T \boldsymbol{\beta}_1, \mathbf{H}^T \boldsymbol{\beta}_2, \mathbf{H}^T \boldsymbol{\beta}_3] \quad (29)$$

$$\dot{\mathbf{r}} = [\dot{r}_1, \dot{r}_2, \dot{r}_3] = [c\dot{\mathbf{H}}^T \boldsymbol{\beta}_1, c\dot{\mathbf{H}}^T \boldsymbol{\beta}_2, c\dot{\mathbf{H}}^T \boldsymbol{\beta}_3] \quad (30)$$

$$\ddot{\mathbf{r}} = [\ddot{r}_1, \ddot{r}_2, \ddot{r}_3] = [c^2\ddot{\mathbf{H}}^T \boldsymbol{\beta}_1, c^2\ddot{\mathbf{H}}^T \boldsymbol{\beta}_2, c^2\ddot{\mathbf{H}}^T \boldsymbol{\beta}_3] \quad (31)$$

Where the output weights $\boldsymbol{\beta} = [\boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \boldsymbol{\beta}_3]$ have been split into three parts, each associated to one of the components of the position. c is a normalization term. **It is worth noting that the equations of motion need to be normalized to learn the solution effectively.** This is common for any application of machine learning, as neural networks do not do well when the learned quantities vary widely and in general, when the inputs and outputs are large. This is especially true with ELM where the activation functions range between -1 and 1 and can easily saturate with excessively large or small inputs.

For this reason, the equations of motion are non-dimensionalized. This is done differently for Keplerian and non-Keplerian orbits. For Keplerian orbits, the equations are non-dimensionalized using a unit of time equal to the total period between the first and the last observation and a unit of length equal to the radius of the Earth. The unit of velocity is obtained by dividing the unit of length by the unit of time. For non-Keplerian orbits, the equations of the CRTBP are non-dimensionalized using the following units:

$$L = 3.85e5 \quad (\text{mean orbital radius of the Moon}) \quad (32)$$

$$T = \frac{2.361e6}{2\pi} \quad (\text{mean orbital period of the Moon}) \quad (33)$$

Time is then further normalized between -1 and 1 as to follow common practice when using ELMs [19]. This is done by mapping the variable t into the new normalized variable z . The normalization term to transform one into the other is:

$$c = \frac{z_f - z_0}{t_f - t_0} \quad (34)$$

where $z_f = 1$ and $z_0 = -1$ and t_f and t_0 depend on the mission profile. We can then switch between dimensional t and non-dimensional z using the following transformation:

$$t = t_0 + \frac{1}{c}(z - z_0) \quad (35)$$

We then use this mapping to approximate the measurement using a variety of measurement models depending on the type of inputs considered. Two combinations of input are considered: 1) range (ρ) and range-rate ($\dot{\rho}$), 2) and right ascension α ($\sin \alpha$, $\cos \alpha$) and declination δ ($\sin \delta$). The loss vector $\mathcal{L}$ is constructed using a combination of the following terms:

$$\begin{aligned} L_{f_i} &= f_i(\mathbf{x}(t), t) \\ L_{\rho} &= \rho - \rho_{NN} \\ L_{\dot{\rho}} &= \dot{\rho} - \dot{\rho}_{NN} \\ L_{\sin \alpha} &= \sin \alpha - \sin \alpha_{NN} \\ L_{\cos \alpha} &= \cos \alpha - \cos \alpha_{NN} \\ L_{\sin \delta} &= \sin \delta - \sin \delta_{NN} \end{aligned} \quad (36)$$

L_{f_i} are the dynamical equations in implicit form while L_{ρ} , $L_{\dot{\rho}}$, $L_{\sin \alpha}$, $L_{\cos \alpha}$ and $L_{\sin \delta}$ are the measurement losses. The subscript NN has been introduced to express the measurements obtained using the output of the neural network (NN), as opposed to the real measurements.

These residuals need to be minimized to solve the OD problem. According to the PINN formalization, the terms L_{ρ} , $L_{\dot{\rho}}$, $L_{\sin \alpha}$, $L_{\cos \alpha}$ and $L_{\sin \delta}$ ensure that the solution is close to the observations, while the term L_{f_i} ensure that the solution abide by the physics of the problem. The residuals are then analytically differentiated with respect to the parameters to learn β . The resulting derivatives are arranged in a matrix representing the Jacobian:

$$J = \frac{dL}{d\beta} \quad (37)$$

The particular expression of the Jacobian depends on the problem and can be found in Sect. 5. Being the problem non-linear, β cannot be obtained with a matrix inversion, as it is usually done for PIELM. Instead, the solution is obtained by solving the non-linear least squares problem using the function *lsqnonlin* in Matlab, which implements the Levenberg–Marquardt algorithm [28, 29]. The optimal weight vector β is then used to recover the solution (position and velocity) at each epoch.

5 Numerical Results and Discussion

The proposed method is tested on different scenarios. The observed satellite is considered on a LEO, a GEO, a Molniya orbit, a Lyapunov, and a halo orbit around the L2 Lagrangian point in the Earth–Moon system. The algorithm is tested for each scenario with range/range-rate and angular measurements. The orbital specifications of the Keplerian and non-Keplerian orbits are given in Tables 1 and 2.

The equations of motion governing the dynamics of the problem are different depending on the case. For LEO, GEO, and Molniya orbits, the dynamical

Table 1 Test orbits (Keplerian)

	a (km)	e	i	ω	Ω	θ_0
LEO	9000	0.05	15	30	45	120
GEO	42,164	0	0	0	0	–
Molniya	26,600	0.74	63.4	270	0	120

Angles are in degrees

Table 2 Test orbits (non-Keplerian)

	Period	x	y	z	$\dot{x}$	$\dot{y}$	$\dot{z}$
Lyapunov	3.4075	1.1790	0	0	0	–0.1422	0
Halo	3.3956	1.1783	0	–0.0490	0	–0.1684	0

Non-dimensional units

Table 3 Ground stations coordinates

	Latitude (°)	Longitude (°)	Elevation (m)
Catalina	32.417	–110.733	2791
Haleakala	20.708	–156.257	3052
Tenerife	28.474	–16.308	2390
Cerro Tololo	–30.169	–70.806	2207
Asiago	45.866	11.5264	1045
South African Large Telescope	–32.376	20.810678	1798

equations are given in Eq. (21), for Lyapunov and halo orbit, they are given in Eq. (22). The measurements scenarios considered are: (1) range (ρ) and range-rate ($\dot{\rho}$), and (2) angular measurements of topocentric right ascension α and declination δ ($\sin \alpha$, $\cos \alpha$, $\sin \delta$). The geographical coordinates of the ground stations are reported in Table 3.

At any given time, the observation is considered to be coming from the observatory whose zenith direction is closest to the observed satellite in the local sky. The observations are generated by integrating the equations of motion described above, adding a gaussian noise to the real range, range rate, right ascension, and declination. The standard deviations of the range and range rate errors are 0.01 m and 0.001 m/s, respectively. The standard deviation on α and δ are 0.5'' instead, chosen as a statistical error for observatories used for astrometric measurements [30]. The observation timespan is considered to be always 1/2 of the orbital period of the target satellite. To showcase the capability of the method, a grid of number of observation (n) and number of neurons (m) is created. The grid is equally spaced between 20 and 300 with step 20 in both directions for keplerian dynamics cases and between 20 and 200 with step of 20 for non-keplerian dynamics cases. The whole algorithm is coded in MATLAB and is run on

a machine with an Intel Core i7-9700 @3.00GHz and 64 GB of RAM. Although the number of neurons/number of samples search is computationally demanding, the network optimization run for a single combination of n and m is in the order of tens of seconds, depending on the size of the problem. Once the network is optimized, the time needed for forward pass through the network is in the order of milliseconds.

5.1 Case 1: Range and Range-Rate

In this case we consider range (ρ) and range-rate ($\dot{\rho}$) as observables. Using the measurement models described in Eq. (36), the loss vector is:

$$\mathbf{L} = [\mathbf{L}_{f_1}, \mathbf{L}_{f_2}, \mathbf{L}_{f_3}, \mathbf{L}_{\rho}, \mathbf{L}_{\dot{\rho}}] \quad (38)$$

The Jacobian in this case is:

$$\mathbf{J} = \frac{d\mathbf{L}}{d\boldsymbol{\beta}} = \begin{bmatrix} \frac{\partial \mathbf{L}_{f_1}}{\partial \beta_1} & \frac{\partial \mathbf{L}_{f_1}}{\partial \beta_2} & \frac{\partial \mathbf{L}_{f_1}}{\partial \beta_3} \\ \frac{\partial \mathbf{L}_{f_2}}{\partial \beta_1} & \frac{\partial \mathbf{L}_{f_2}}{\partial \beta_2} & \frac{\partial \mathbf{L}_{f_2}}{\partial \beta_3} \\ \frac{\partial \mathbf{L}_{f_3}}{\partial \beta_1} & \frac{\partial \mathbf{L}_{f_3}}{\partial \beta_2} & \frac{\partial \mathbf{L}_{f_3}}{\partial \beta_3} \\ \frac{\partial \mathbf{L}_{\rho}}{\partial \beta_1} & \frac{\partial \mathbf{L}_{\rho}}{\partial \beta_2} & \frac{\partial \mathbf{L}_{\rho}}{\partial \beta_3} \\ \frac{\partial \mathbf{L}_{\dot{\rho}}}{\partial \beta_1} & \frac{\partial \mathbf{L}_{\dot{\rho}}}{\partial \beta_2} & \frac{\partial \mathbf{L}_{\dot{\rho}}}{\partial \beta_3} \end{bmatrix} \quad (39)$$

The performance is evaluated in terms of Root Mean Squared Error (RMSE) on the estimated states. The RMSE is calculated along the whole trajectory as:

$$RMSE = \frac{1}{6} \sum_{i=1}^6 \sqrt{\frac{\sum_{t=1}^T (\hat{x}_i(t) - x_i(t))^2}{T}} \quad (40)$$

where $\hat{x}_i(t)$ is the i -th component of the estimated state at time t , $x_i(t)$ is the real state and T is the total number of steps. It should be noted that contrarily to what is normally done in typical machine learning applications, in this case the focus is put on the accuracy of the model on the training data and not on the generalization capabilities of the model. This is common to many physics informed neural networks applications, where the model is trained to solve a specific problem, identified by a specific set of DEs and a measurements.

The results of the test campaign are shown in Fig. 3, where the inverse of the RMSE is plotted against the parameters. Using this visualization, the combination that achieves the lowest RMSE corresponds to the highest peak.

It is common in all cases that a particular combination of n and m stands out from the rest, giving a significantly lower RMSE. The number of samples has the

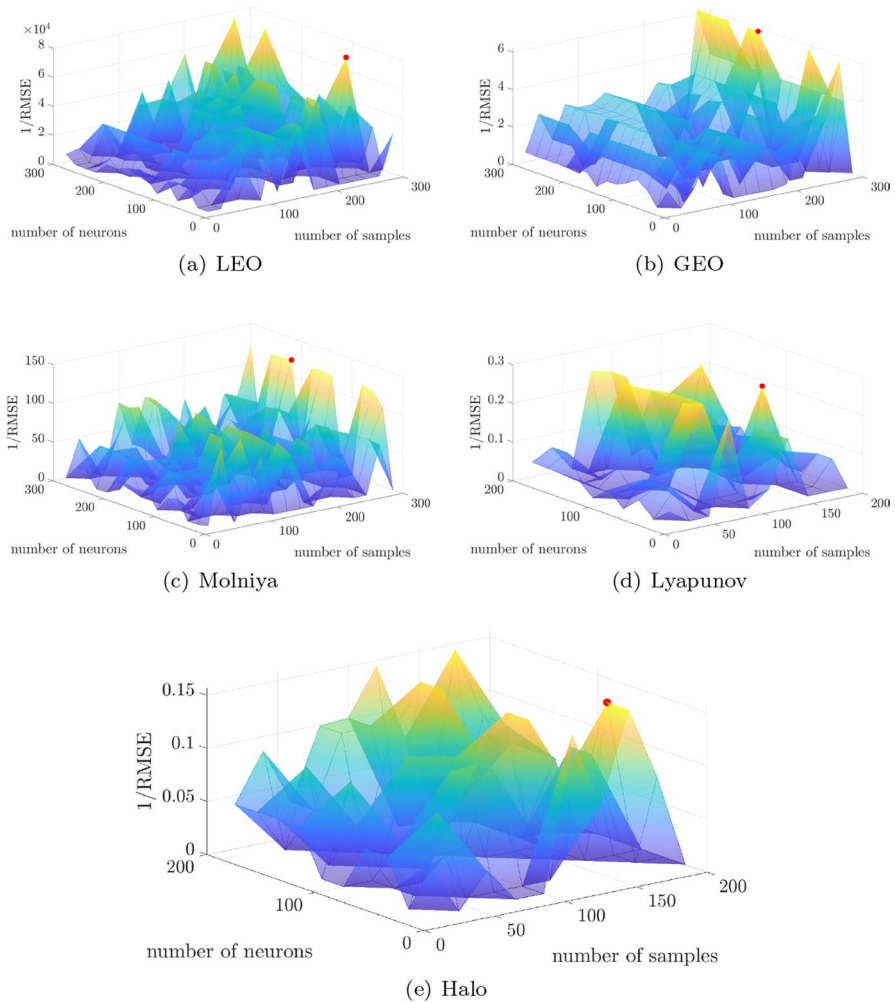


Fig. 3 Neuron number selection for range and range-rate cases. The red dot indicates the best combination of number of samples and number of neurons (Color figure online)

biggest impact on the performance, with higher n values giving smaller RMSE in general. The number of neurons does not have a significant impact on performance, except for very low m , which generally gives poor performance. The errors in terms of RMSE along the trajectory using the combination that gives the smallest error are shown in Fig. 4.

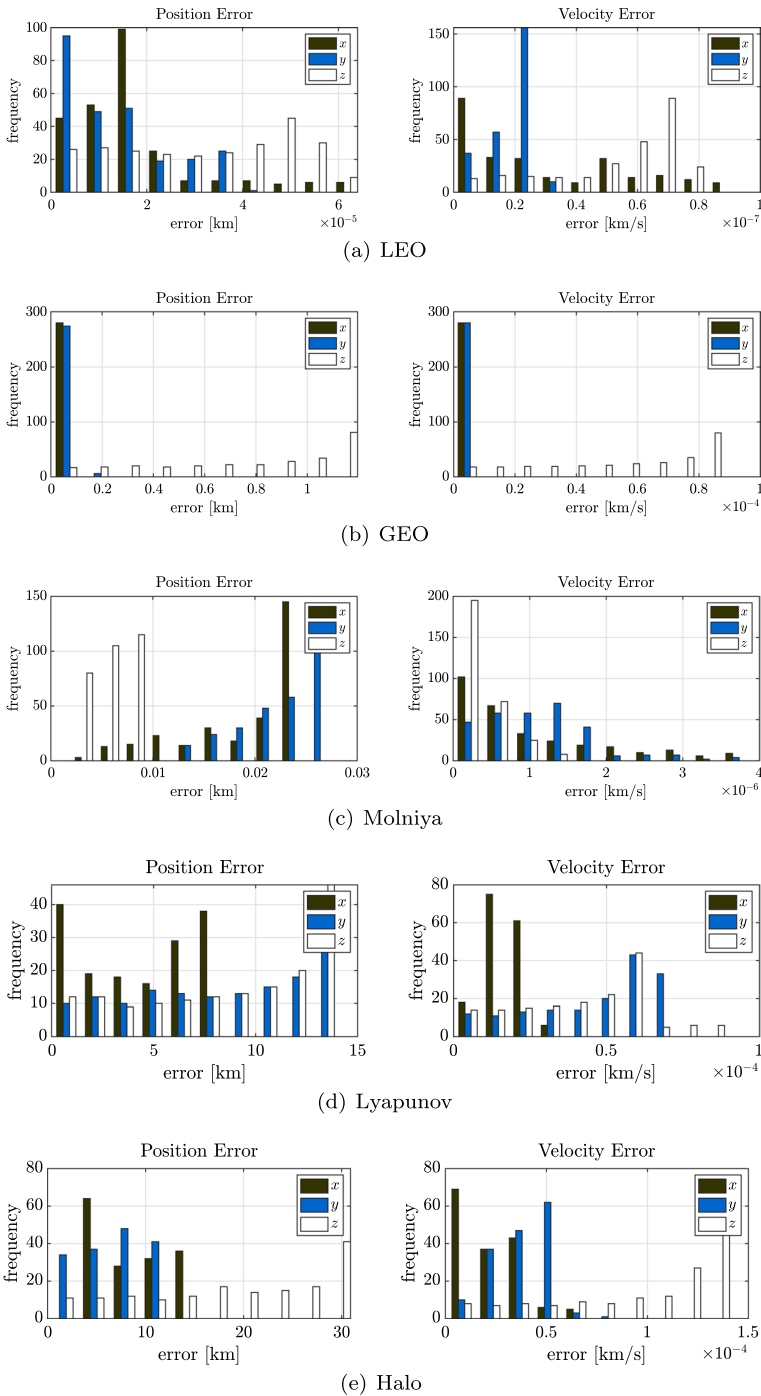


Fig. 4 State errors for range and range-rate cases

Table 4 Range and range-rate

	LEO	GEO	Molniya
PIOD			
n	260	280	300
m	60	180	220
<i>a</i>	3.776×10^{-5}	3.510×10^{-3}	2.714×10^{-3}
<i>e</i>	3.502×10^{-9}	2.006×10^{-6}	3.274×10^{-9}
<i>i</i>	5.778×10^{-7}	1.174×10^{-3}	2.188×10^{-5}
Ω	4.261×10^{-7}	–	4.941×10^{-6}
ω	1.072×10^{-6}	–	5.225×10^{-5}
θ	1.310×10^{-6}	–	4.508×10^{-6}
Batch LS			
<i>a</i>	1.150×10^{-6}	8.964×10^{-5}	3.725×10^{-5}
<i>e</i>	2.347×10^{-11}	5.489×10^{-7}	4.837×10^{-9}
<i>i</i>	6.333×10^{-9}	5.193×10^{-4}	1.117×10^{-6}
Ω	3.707×10^{-7}	–	1.587×10^{-6}
ω	1.182×10^{-7}	–	8.365×10^{-6}
θ	2.903×10^{-7}	–	7.449×10^{-7}

Orbital parameters error for Keplerian orbits. Distances are in km and angles in degrees

The RMSE show that the most challenging cases are the GEO, Molniya and non-keplerian orbits. For the GEO orbit, the largest error is observed on the z component of position and velocity, indicating that the network is struggling to lay the estimated trajectory on the equatorial plane. For non-keplerian orbits, although the absolute error is generally greater than the keplerian cases, when related to the average distance of the target, the results are comparable. Moreover, the resulting error on the orbital parameters for the Keplerian orbits is shown in Table 4 and compared with the classical batch least-squares algorithm described in Sect. 2. The method's performance in terms of orbital parameter estimation is comparable to the batch least-squares, the latter being slightly more accurate.

5.2 Case 2: Right-Ascension and Declination

In this case, the observations are trigonometric functions of the topocentric right ascension (α) and declination (δ). Three observations are enough to characterize the position of the satellite completely. We considered using both sine and cosine of the two angles but we did not observe a significant difference in the results. The loss in this case is:

$$L = [L_{f_1}, L_{f_2}, L_{f_3}, L_{\sin \alpha}, L_{\cos \alpha}, L_{\sin \delta}] \quad (41)$$

The Jacobian then is:

$$J = \frac{dL}{d\beta} = \begin{bmatrix} \frac{\partial L_{f_1}}{\partial \beta_1} & \frac{\partial L_{f_1}}{\partial \beta_2} & \frac{\partial L_{f_1}}{\partial \beta_3} \\ \frac{\partial L_{f_2}}{\partial \beta_1} & \frac{\partial L_{f_2}}{\partial \beta_2} & \frac{\partial L_{f_2}}{\partial \beta_3} \\ \frac{\partial L_{f_3}}{\partial \beta_1} & \frac{\partial L_{f_3}}{\partial \beta_2} & \frac{\partial L_{f_3}}{\partial \beta_3} \\ \frac{\partial L_{\sin \alpha}}{\partial \beta_1} & \frac{\partial L_{\sin \alpha}}{\partial \beta_2} & \frac{\partial L_{\sin \alpha}}{\partial \beta_3} \\ \frac{\partial L_{\cos \alpha}}{\partial \beta_1} & \frac{\partial L_{\cos \alpha}}{\partial \beta_2} & \frac{\partial L_{\cos \alpha}}{\partial \beta_3} \\ \frac{\partial L_{\sin \delta}}{\partial \beta_1} & \frac{\partial L_{\sin \delta}}{\partial \beta_2} & \frac{\partial L_{\sin \delta}}{\partial \beta_3} \\ \frac{\partial L}{\partial \beta_1} & \frac{\partial L}{\partial \beta_2} & \frac{\partial L}{\partial \beta_3} \end{bmatrix} \quad (42)$$

Figure 5 shows testing campaign results. The number of samples clearly has the the most influence on the error, with higher n leading to lower RMSE in most cases, except for the Lyapunov orbit. In these cases, the number of neurons m do not play a significant role as indicated by generally flat curves in the m dimension.

Figure 6 shows the state error for all cases. The performance is comparable for all cases. The Keplerian orbits are also compared with classical batch least squares. Table 5 shows the results. In this case, PIOD compares favourably with batch least-squares, with slightly better performance for LEO and comparable performance in the other cases. Moreover, Table 6 shows the optimization times for all the selected cases.

It is important to point out some key advantages of the PIOD method presented. Although it is powerful and the estimation errors are comparable with batch least squares, the strength of the method resides in the fact that it does not require any initial guess to work. The only parameter that has to be initialized are the weights and biases of the neural network. These, thanks to the ELM formalization, are initialized at random and never changed during the optimization process. Another key advantage is that PIOD estimates the entire trajectory between the first and last observation at once, without integration. This is possible due to the nature of the physics informed neural network, which creates a general representation of the input–output relationship according to the problem’s physics. The network can then be sampled at any point within the span of the observation window.

5.3 Generalization Test

The networks selected using the method described in the previous sections were then tested on a test data-set for each case. This was done to test the generalization capabilities of the method by querying the model on test points that have never been seen by the network. In order to do so, 1000 equally distributed points were sampled on each trajectory within the same interval used for training. A forward pass across the trained network was performed for each input and the resulting estimated position and velocity was recorded. The results in terms of position and velocity error closely match the results shown in Figs. 4 and 6. This is in accordance with expectations since using the physics to regularize the NN training ensures that the

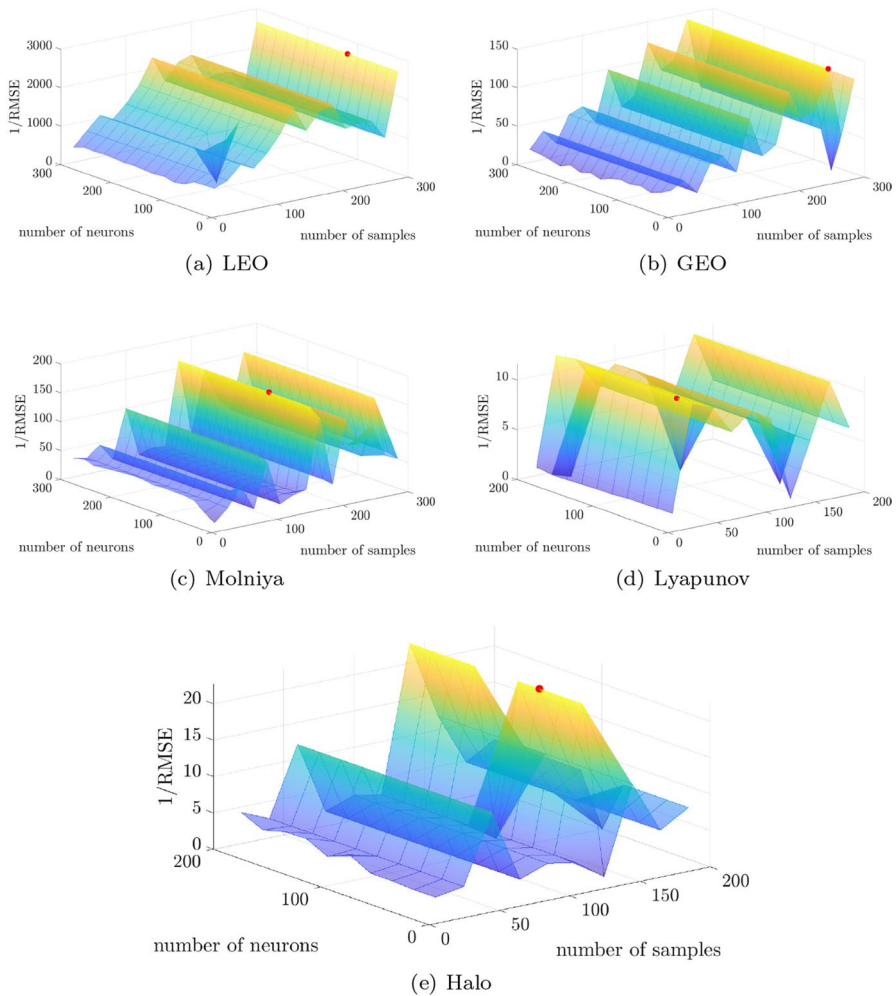


Fig. 5 Neuron number selection for the right-ascension/declination case. The red dot indicates the best combination of number of samples and number of neurons (Color figure online)

estimated state is physically sound no matter the query point. Given the similarity of these results with the results already shown and for brevity, the resulting plots are not reported. Nevertheless, it is interesting to compare these results with the least squares solution, obtained by integrating the estimated initial state forward in time and reporting the error at each test point. Figures 7 and 8 show the errors along the trajectory. The method performs on par with batch least squares, with errors being in the same order of magnitude of batch least squares in most cases. A slight advantage for batch least squares was observed on average when using range and range rate observations.

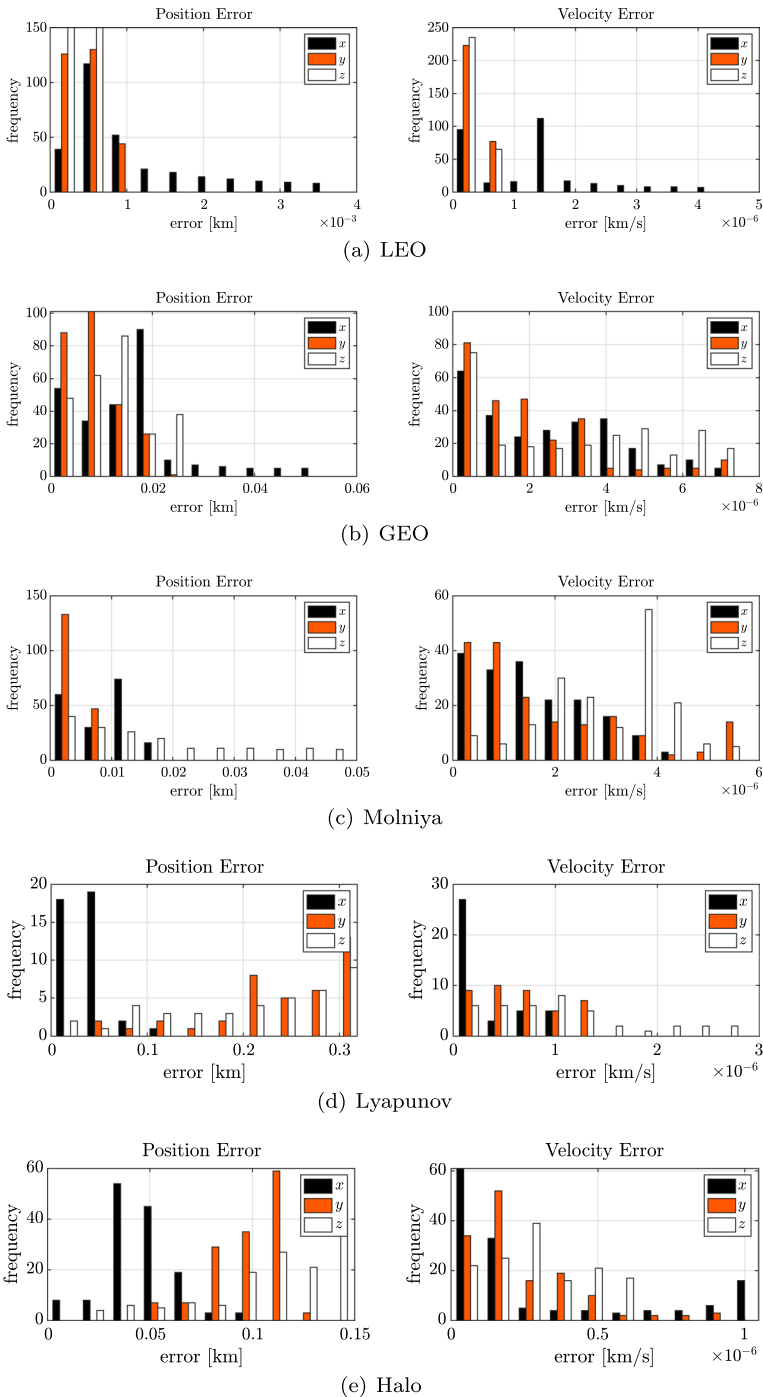


Fig. 6 State error for the right-ascension/declination case

Table 5 Right ascension and declination

	LEO	GEO	Molniya
PIOD			
<i>n</i>	300	260	180
<i>m</i>	120	20	120
<i>a</i>	6.859×10^{-4}	9.681×10^{-2}	9.142×10^{-3}
<i>e</i>	4.410×10^{-8}	2.362×10^{-6}	3.024×10^{-8}
<i>i</i>	3.585×10^{-6}	1.322×10^{-4}	1.075×10^{-4}
Ω	3.741×10^{-5}	–	4.087×10^{-5}
ω	1.831×10^{-5}	–	1.316×10^{-4}
θ	2.494×10^{-5}	–	2.782×10^{-5}
Batch LS			
<i>a</i>	2.113×10^{-3}	3.914×10^{-3}	8.415×10^{-3}
<i>e</i>	2.347×10^{-7}	8.428×10^{-8}	4.230×10^{-7}
<i>i</i>	1.643×10^{-6}	1.198×10^{-5}	8.894×10^{-6}
Ω	1.493×10^{-4}	–	1.869×10^{-7}
ω	1.618×10^{-5}	–	7.917×10^{-6}
θ	1.404×10^{-4}	–	8.815×10^{-6}

Orbital parameters error for Keplerian orbits. Distances are in km and angles in degrees

Table 6 Optimization times

	LEO	GEO	Molniya	Lyapunov	Halo
Range/range-rate	3.751	28.874	12.402	28.306	87.065
RA/DEC	6.649	2.566	10.206	7.320	30.3427

Times reported only for selected solution in seconds

5.4 Test with Simulated Higher Fidelity Data

The algorithm has been tested on more realistic observation data for the Keplerian dynamics orbits. The observations are generated using the General Mission Analysis Tool (GMAT). The model accounts for the oblateness of the Earth, the atmospheric drag due to friction, and SRP. The gravity model is based on the NASA Goddard JGM-2 model with degree and order set to 10. The drag model is based on the Jachia-Roberts atmosphere model, assuming a spherical spacecraft to neglect attitude. The SRP model also assumes a spherical spacecraft. The spacecraft's trajectory is simulated using the internal GMAT Runge-Kutta89¹ adaptive step integrator. The data is reported every 10 s and is then resampled at different intervals depending on the case. For the LEO, the interval between the observations is 10 s, for GEO and Molniya it is 300 s. The observations are generated using the simulated windows of visibility from the observatories described in the previous section. This accurate simulation was run to test the method on higher fidelity simulated data that are more similar to real-world data. The errors on the orbital parameters are given in Table 7.

¹ <http://gmats.sourceforge.net/docs/R2020a/html/Propagator.html>.

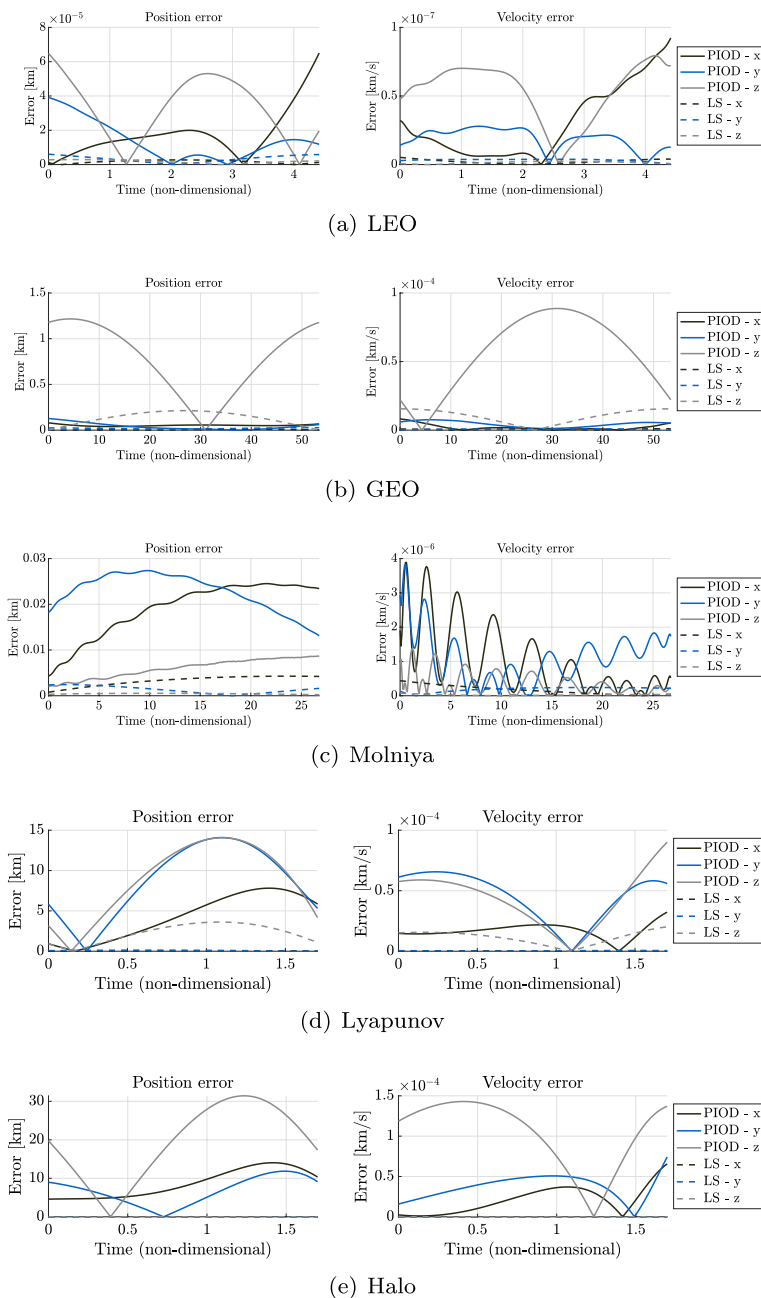


Fig. 7 Error along trajectory. Range/range-rate

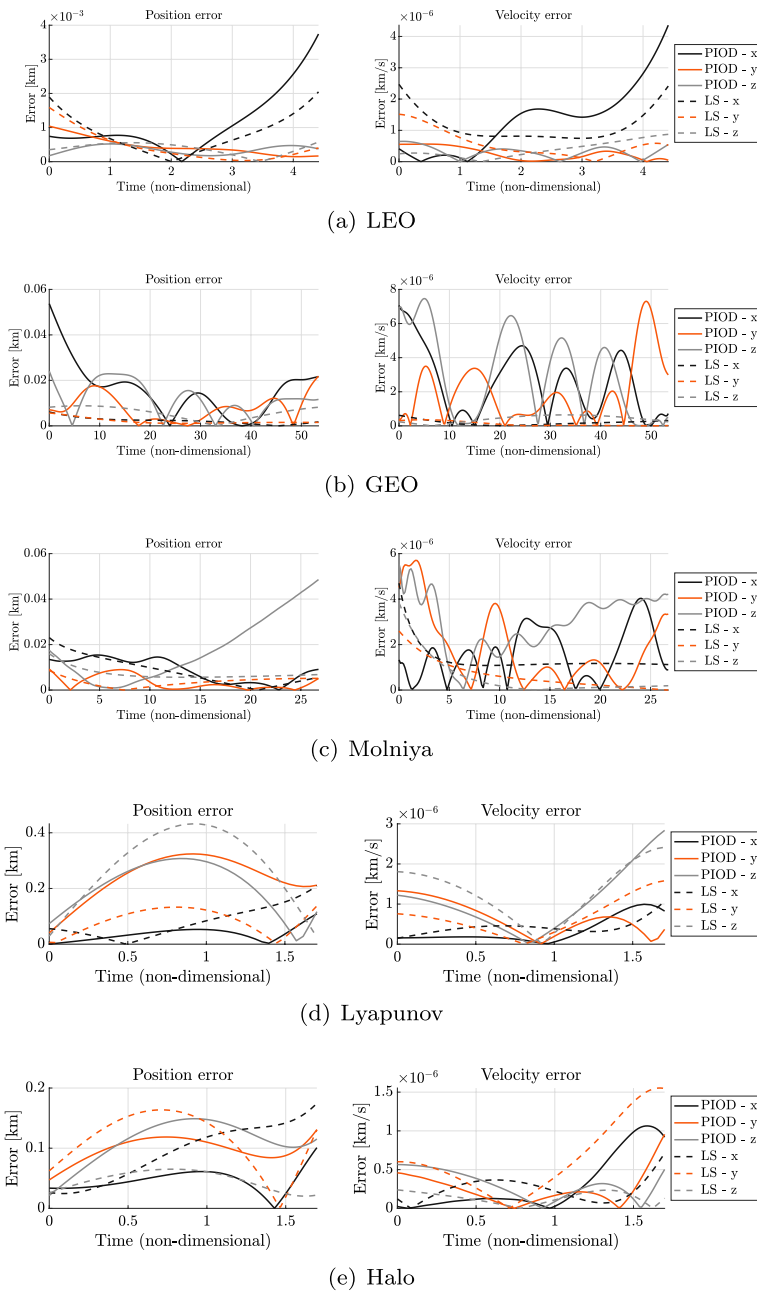


Fig. 8 Error along trajectory. Right-ascension/declination

Table 7 Test with GMAT data, angle-only observations. Orbital parameters error for Keplerian orbits. Distances are in km and angles in degrees

	LEO	GEO	Molniya
PINN-OD			
<i>m</i>	40	200	20
<i>a</i>	1.194×10^1	2.033×10^0	1.161×10^0
<i>e</i>	1.091×10^{-3}	3.456×10^{-5}	1.972×10^{-5}
<i>i</i>	9.859×10^{-3}	6.369×10^{-5}	1.583×10^{-3}
Ω	5.984×10^{-2}	–	3.584×10^{-3}
ω	5.344×10^{-2}	–	1.336×10^{-3}
θ	1.121×10^{-1}	–	4.284×10^{-3}
Batch LS			
<i>a</i>	9.784×10^0	2.104×10^0	1.782×10^0
<i>e</i>	8.603×10^{-4}	3.557×10^{-5}	3.902×10^{-5}
<i>i</i>	2.711×10^{-4}	5.913×10^{-5}	3.349×10^{-3}
Ω	4.791×10^{-1}	–	4.315×10^{-3}
ω	1.979×10^{-1}	–	3.920×10^{-3}
θ	2.844×10^{-1}	–	6.323×10^{-3}

The results are in accordance with the expectations: due to the perturbations in the equations of motion, the network cannot estimate the trajectory with the same level of accuracy as in the previous section since it has been trained using unperturbed equations. The difference can be mainly observed in the Molniya orbit case, which has the highest error. The results are also compared with the solution obtained using classical batch LS. In this case, the methods perform similarly in all cases.

Although this test gives insight into the robustness of the method to unmodelled dynamics, other natural perturbations and commanded actions, such as station-keeping maneuvers in cislunar space, can be included in the model. Future work can include these terms in the equations of motion and consequently in the loss function, which could increase the method's accuracy in a real-world scenario. In case the additional forcing terms are unknown, these could be estimated alongside the state by introducing additional terms to the Jacobian, exploiting the capability physics informed neural networks to perform parameters estimation and problem inversion [31].

5.5 Batch Least Squares Sensitivity Analysis to Initial Guess

The proposed method has a notable strength in its capability to converge even with random initialization of network parameters. In contrast, batch least squares requires a reasonably close approximation of the actual satellite state to achieve convergence. To demonstrate this behavior, the sensitivity of batch least squares convergence to the initial condition error has been examined. For each combination of observations (range/range-rate and right ascension/declination) and orbit (LEO, GEO, Molniya, Lyapunov, and Halo), batch least squares has been executed 100 times, gradually increasing the error on the initial guess. The sensitivity to both velocity and position error has been

tested by first keeping the position constant and varying the velocity, and then fixing the velocity while allowing the position to vary. The error vector has been added to the real initial state of each trajectory. The magnitude of the vector has been increased at each test point within suitable boundaries based on the specific case, while the direction has been randomized for each test point. This analysis provided insights into the convergence sensitivity to both the magnitude and direction of the error. Convergence was considered achieved if the norm of the step fell below a small threshold. When convergence was attained, the method demonstrated rapid convergence in a few iterations, enabling easy detection of non-converging cases where the solution tended to diverge until a maximum number of steps was reached. The results, indicating converged and non-converged cases are presented in Figs. 9 and 10. The magnitude of error in initial velocity/position (Δ_v , Δ_r) is expressed as a percentage of the real initial velocity/position magnitude, while θ is the angle between the error vector and the real initial velocity/position direction.

While there is not a defined threshold above which the method becomes unstable, batch least squares experiences instability in all cases. The method tends to diverge more often as the angular distance of the error deviates from 90° , which indicates that an error orthogonal to the actual state is generally the most favorable condition for convergence. This finding is especially problematic when considering angle only observations since, without access to ranging measurements, the error is indeed mostly aligned with the slant vector, which, especially for far away targets such as in cislunar space, is almost aligned with the position vector. Moreover, a clear difference in performance is observable between keplerian and non-keplerian cases. The error magnitude in position or velocity required for batch least squares to diverge is much lower in non-keplerian cases, where errors above 1% in position and 5% in velocity are enough for convergence failure.

The instability experienced by batch least squares is in stark contrast with the remarkable convergence capabilities of PIOD. It should be noted that in all these cases, PIOD converges using a random initialization of parameters as demonstrated in the previous sections. This effectively removes the need for any previous information about the target state, which is extremely valuable, especially when dealing with non-cooperative targets. Moreover, the poorer performance of batch least squares in cislunar environment makes a compelling argument for the viability of new methods such as PIOD, which can enable better state estimation for non-cooperative cislunar objects. Indeed, categorizing and tracking those objects is becoming a strong need to the scientific community due to the increased interest and number of missions towards the cislunar environment.

6 Conclusions

The proposed method combines the power of machine learning and least-squares OD techniques to create a new orbit determination method named PIOD (PIOD). It is based on physics informed random SLFNs is used to approximate the latent solution of an ODE or PDE. This works by penalizing the loss function of the optimization method used to obtain the network parameters with the dynamical system. So

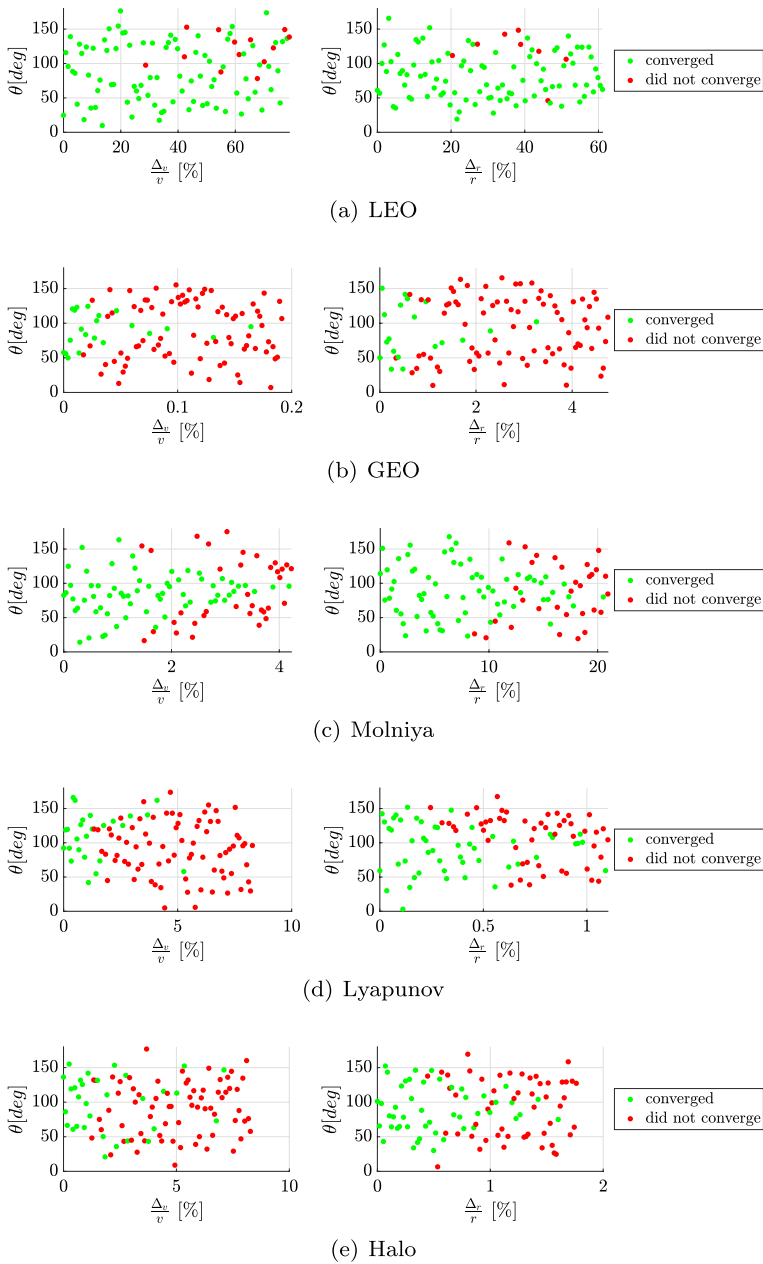


Fig. 9 Batch least squares convergence sensitivity analysis. Range/range-rate

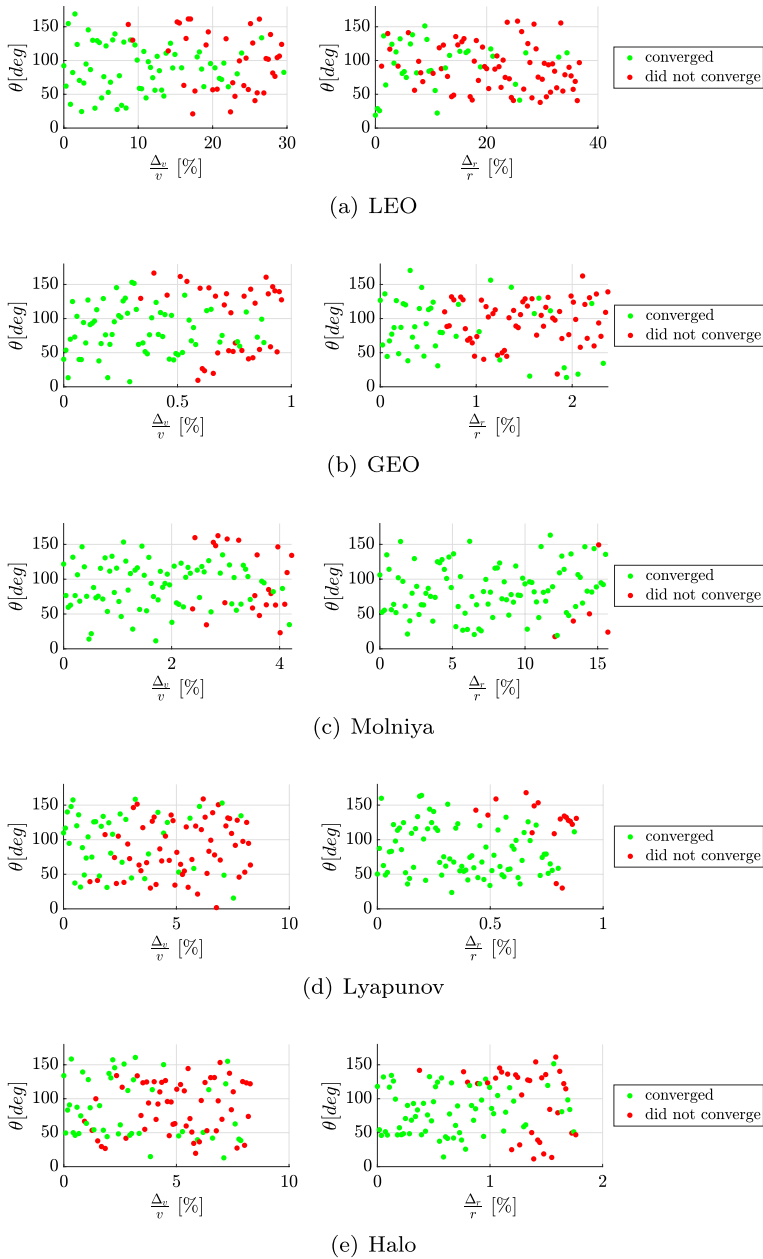


Fig. 10 Batch least squares convergence sensitivity analysis. Right-ascension/declination

the loss is composed of a term related to the measurements, calculated as the difference between the position output of the net and its derivatives and the noisy observations coming from ground observations. The measurement loss is then penalized with the dynamics residual based on the equations of motion.

The method can estimate the state of a satellite both in Keplerian and non-Keplerian dynamics, using either range/range-rate or right-ascension/declination measurements. Its performance is on par with classical batch least-squares in all cases. Nevertheless, when compared to the latter, PIOD has some peculiar advantages. Since PIOD estimates the entire observed trajectory at once, there is no need for integration, which removes additional sources of numerical error. Moreover, the particular structure of PIOD based on ELM allows it to provide an accurate estimate without needing a precise initial guess. The only parameters to initialize are input weights and biases, which as per ELM theory, are initialized at random. This essentially removes the dependence of the solution on the initial guess. This is in stark contrast with batch least squares, which shows convergence sensitivity to initial guess, especially when estimating the state of satellites in cislunar space, that is becoming increasingly critical for the future of space exploration. Overall, this paper shows that physics informed neural networks are a powerful tool that can be employed for OD, which performs on par or better than existing least-squares techniques, without requiring any initial guess. Moreover, the flexibility of PINNs allows the method to be extended further using a more refined parameter selection algorithm, as well as adding functionality to account for maneuvering targets and perturbations.

Data Availability Data source employed in this paper can be made available by the authors upon reasonable request.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Gauss, C.F.: *Theoria motus corporum coelestium in sectionibus conicis solem ambientium*, vol. 7. FA Perthes, Gotha (1877)
2. Escobal, P.: *Methods of Orbit Determination*. Wiley, New York (1970)
3. Gooding, R.H.: A new procedure for the solution of the classical problem of minimal orbit determination from three lines of sight. *Celest. Mech. Dyn. Astron.* **66**(4), 387–423 (1996)
4. Crassidis, J.L., Junkins, J.L.: *Optimal Estimation of Dynamic Systems*. CRC Press, Boca Raton (2011)
5. Schutz, B., Tapley, B., Born, G.H.: *Statistical Orbit Determination*. Elsevier, Amsterdam (2004)
6. Vallado, D.A.: *Fundamentals of Astrodynamics and Applications*, vol. 12. Springer, Dordrecht (2001)
7. Brown, R.G., Hwang, P.Y.C.: *Introduction to Random Signals and Applied Kalman Filtering: With MATLAB Exercises*, vol. 4. Wiley, New York (2012)
8. Lefebvre, T., Bruyninckx, H., De Schutter, J.: Kalman filters for non-linear systems: a comparison of performance. *Int. J. Control* **77**(7), 639–653 (2004)
9. Fan, L., Wehbe, Y.: Extended kalman filtering based real-time dynamic state and parameter estimation using pmu data. *Electr. Power Syst. Res.* **103**, 168–177 (2013)
10. Lefebvre, T., Bruyninckx, H., De Schutter, J.: A the linear regression kalman filter. In: *Nonlinear Kalman Filtering for Force-Controlled Robot Tasks*, pp. 205–210. Springer, Berlin (2005)
11. Christian, J.A., Lightsey, E.G.: Review of options for autonomous cislunar navigation. *J. Spacecr. Rockets* **46**(5), 1023–1036 (2009)
12. Miller, W.T., Werbos, P.J., Sutton, R.S.: *Neural Networks for Control*. MIT, Cambridge (1995)

13. Furfaro, R., Linares, R., Jah, M.K., Gaylor, D.: Mapping sensors measurements to the resident space objects behavior energy and state parameters space via extreme learning machines. In: International Astronautical Congress (2016)
14. Sharma, S.: Machine Learning Applications in Spacecraft State and Environment Estimation. PhD thesis (2018)
15. Lee, B., Kim, W.-G., Lee, J., Hwang, Y.: Machine learning approach to initial orbit determination of unknown leo satellites. In: 2018 SpaceOps Conference, p. 2566 (2018)
16. Peng, H., Bai, X.: Artificial neural network-based machine learning approach to improve orbit prediction accuracy. *J. Spacecr. Rockets* **55**(5), 1248–1260 (2018)
17. Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019)
18. Schmidt, W.F., Kraaijveld, M.A., Duin, R.P.W. et al.: Feed forward neural networks with random weights. In: International Conference on Pattern Recognition, p. 1. IEEE Computer Society Press, Washington (1992)
19. Huang, G.-B., Zhu, Q.-Y., Siew, C.-K.: Extreme learning machine: theory and applications. *Neurocomputing* **70**(1–3), 489–501 (2006)
20. Dwivedi, V., Srinivasan, B.: Physics informed extreme learning machine (PIELM)—a rapid method for the numerical solution of partial differential equations. *Neurocomputing* **391**, 96–118 (2020)
21. Mao, Z., Jagtap, A.D., Karniadakis, G.E.: Physics-informed neural networks for high-speed flows. *Comput. Methods Appl. Mech. Eng.* **360**, 112789 (2020)
22. Sahli Costabal, F., Yang, Y., Perdikaris, P., Hurtado, D.E., Kuhl, E.: Physics-informed neural networks for cardiac activation mapping. *Front. Phys.* **8**, 42 (2020)
23. Mortari, D.: The theory of connections: connecting points. *Mathematics* **5**(4), 57 (2017)
24. Schiassi, E., Furfaro, R., Leake, C., De Florio, M., Johnston, H., Mortari, D.: Extreme theory of functional connections: a fast physics-informed neural network method for solving ordinary and partial differential equations. *Neurocomputing* **457**, 334–356 (2021)
25. Leake, C., Mortari, D.: Deep theory of functional connections: a new method for estimating the solutions of partial differential equations. *Mach. Learn. Knowl. Extract.* **2**(1), 37–55 (2020)
26. Huang, G.-B., Chen, L., Siew, C.K., et al.: Universal approximation using incremental constructive feedforward networks with random hidden nodes. *IEEE Trans. Neural Netw.* **17**(4), 879–892 (2006)
27. Koon, W.S., Lo, M.W., Marsden, J.E., Ross, S.D.: Dynamical systems, the three-body problem and space mission design. In: *Equadiff 99: (In 2 Volumes)*, pp. 1167–1181. World Scientific, Singapore (2000)
28. Levenberg, K.: A method for the solution of certain non-linear problems in least squares. *Q. Appl. Math.* **2**(2), 164–168 (1944)
29. Marquardt, D.W.: An algorithm for least-squares estimation of nonlinear parameters. *J. Soc. Ind. Appl. Math.* **11**(2), 431–441 (1963)
30. Vereš, P., Farnocchia, D., Chesley, S.R., Chamberlin, A.B.: Statistical analysis of astrometric errors for the most productive asteroid surveys. *Icarus* **296**, 139–149 (2017)
31. Schiassi, E., De Florio, M., D'Ambrosio, A., Mortari, D., Furfaro, R.: Physics-informed neural networks and functional interpolation for data-driven parameters discovery of epidemiological compartmental models. *Mathematics* **9**(17), 2069 (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.